

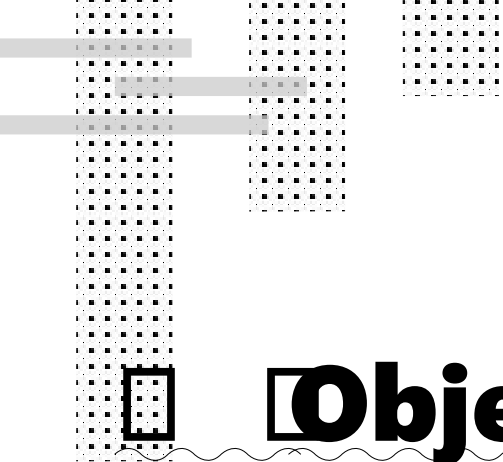


LENGUAJES Y
CIENCIAS DE LA
COMPUTACIÓN
UNIVERSIDAD DE MÁLAGA

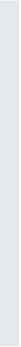
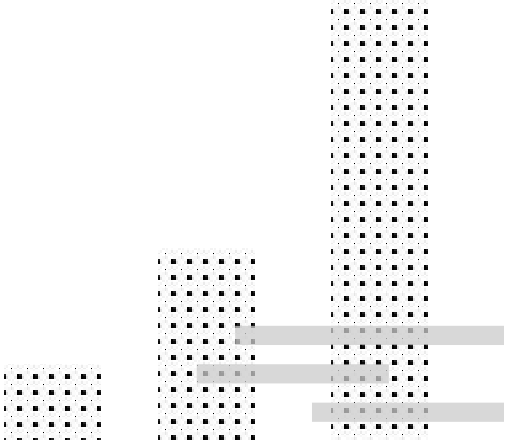
Análisis y Diseño de Aplicaciones

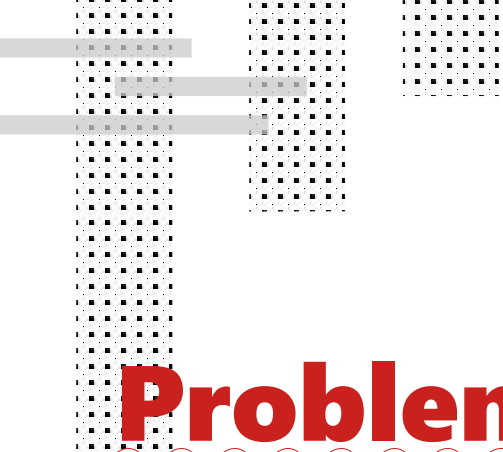
Introducción al Ciclo de Vida del Software

Grado en Ingeniería Informática
2025-2026



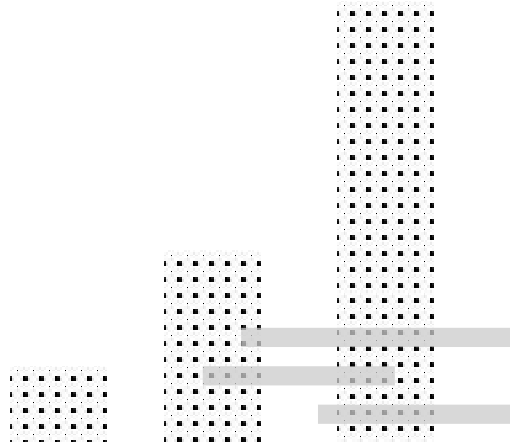
Objetivos de la sesión

- 
- Comprender qué es el ciclo de vida del software
 - Conocer sus principales fases y modelos
 - Identificar el papel del análisis y diseño dentro del ciclo completo
- 
- 



Problemas del software

Complejidad en el Desarrollo de Software

- 
- No es solo escribir código funcional
 - Cambios pequeños pueden causar errores grandes
 - Requiere coordinar personas, tareas y tecnología
 - Muchos niveles: requisitos, diseño, pruebas, mantenimiento
 - Alto número de interdependencias entre módulos
 - Difícil predecir el impacto de cada decisión
 - La complejidad crece con cada nueva funcionalidad

Problemas del software

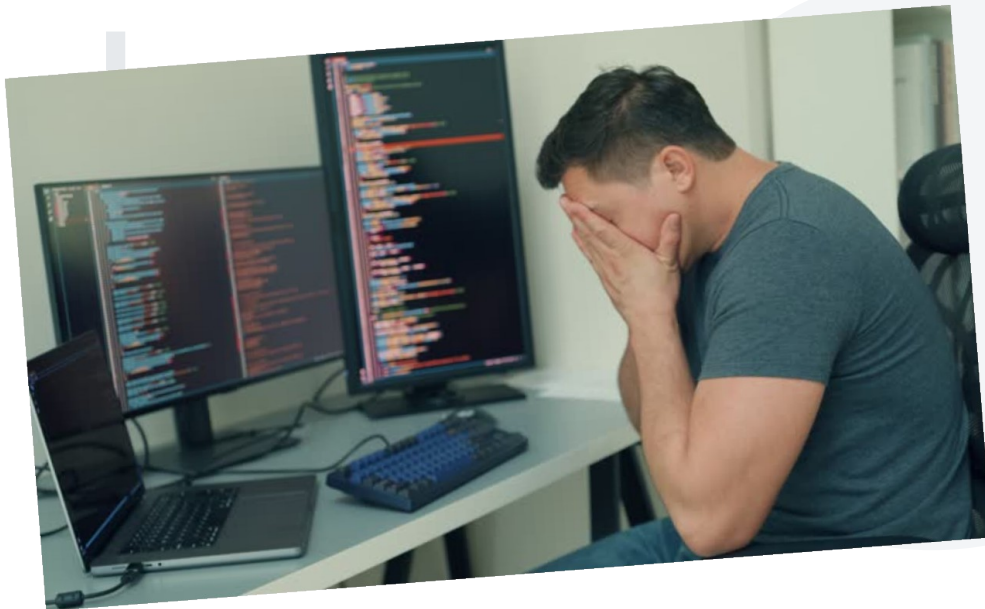
Complejidad en el Desarrollo de Software



No es **solo** escribir **código funcional**

Problemas del software

Complejidad en el Desarrollo de Software



Cambios
pequeños



Errores
grandes

Problemas del software

Complejidad en el Desarrollo de Software

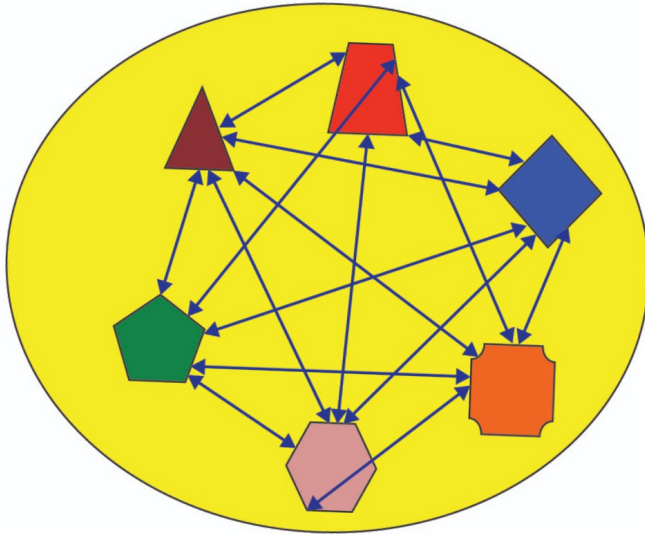


Hay que coordinar:

- Personas
- Tareas
- Tecnología

Problemas del software

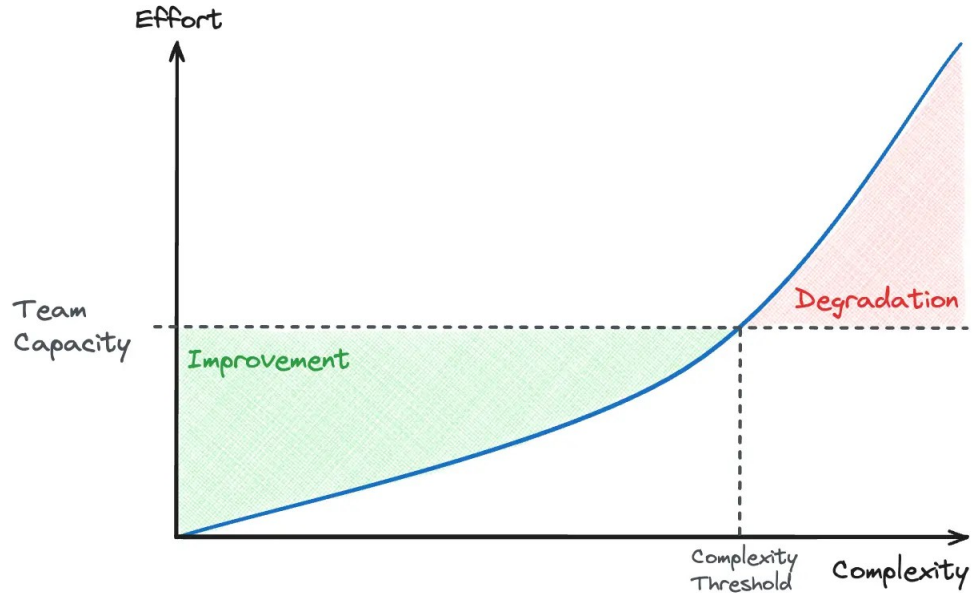
Complejidad en el Desarrollo de Software



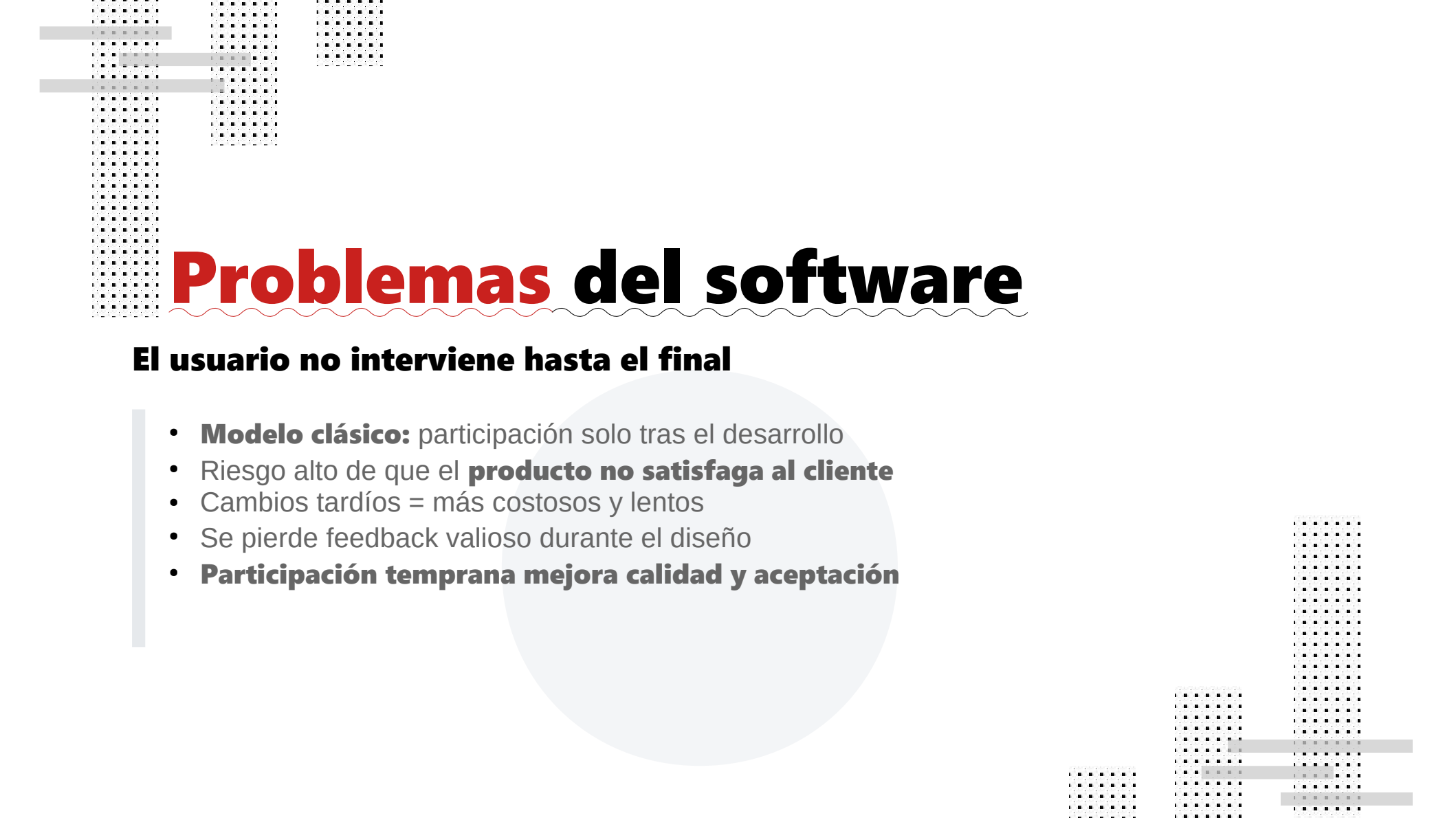
- Alto número de interdependencias entre módulos
- Difícil predecir el impacto de cada decisión

Problemas del software

Complejidad en el Desarrollo de Software



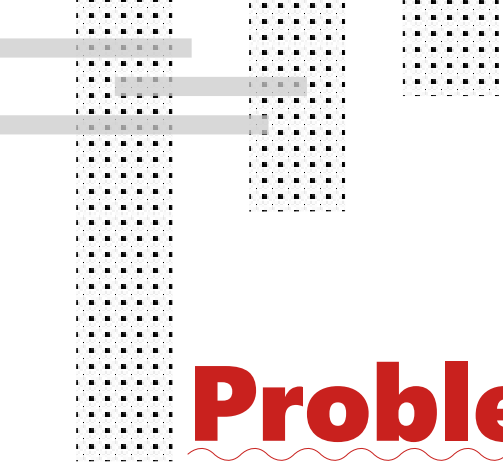
La **complejidad crece** con cada **nueva funcionalidad**



Problemas del software

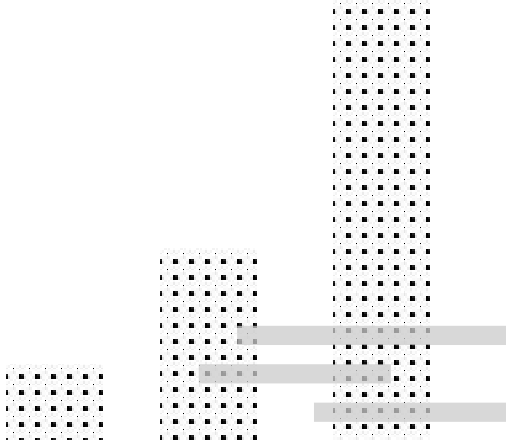
El usuario no interviene hasta el final

- **Modelo clásico:** participación solo tras el desarrollo
- Riesgo alto de que el **producto no satisfaga al cliente**
- Cambios tardíos = más costosos y lentos
- Se pierde feedback valioso durante el diseño
- **Participación temprana mejora calidad y aceptación**



Problemas del software

Cambios durante la construcción o tras la instalación

- 
- El cambio **es inevitable** en cualquier proyecto
 - Nuevas necesidades pueden surgir en desarrollo
 - Errores se descubren tarde, incluso en producción
 - Software debe ser **flexible y evolutivo**
 - Procesos rígidos dificultan adaptaciones rápidas
 - **Anticipar y gestionar cambios = clave del éxito**



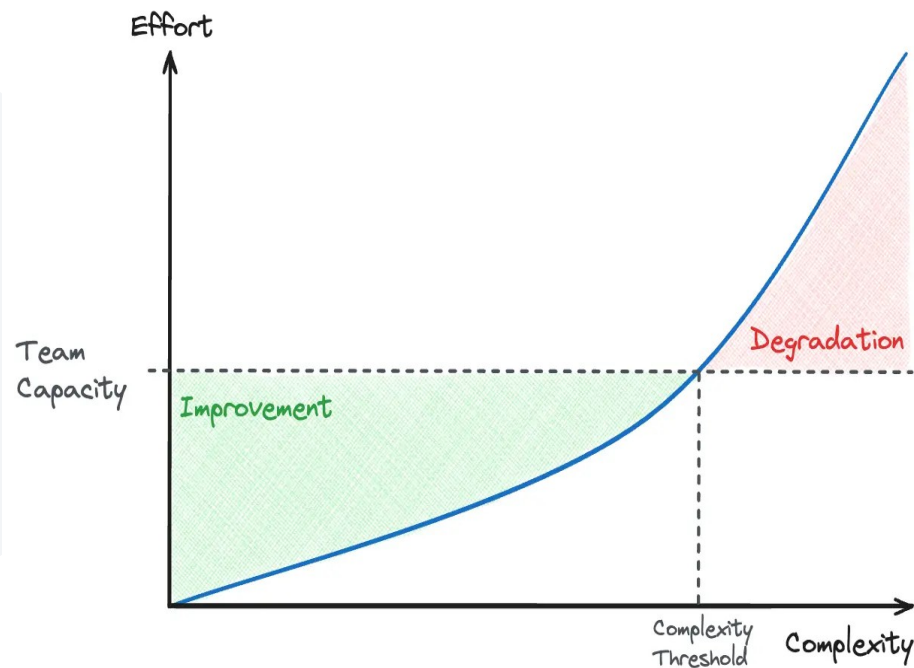
Requisitos de técnicas de desarrollo de software

Minimizar la complejidad de los sistemas

Requisitos de técnicas de desarrollo de software

Minimizar la complejidad de los sistemas

- **Sistemas simples** son más **fáciles de mantener**
- Menos errores, menos coste, más comprensión
- Diseña con modularidad.
- Evita soluciones innecesariamente sofisticadas
- Documenta lo esencial, no lo obvio
- Usa estándares y patrones conocidos
- Menos es más: simplicidad es distinto de debilidad
- Simplificar es un objetivo, no una renuncia

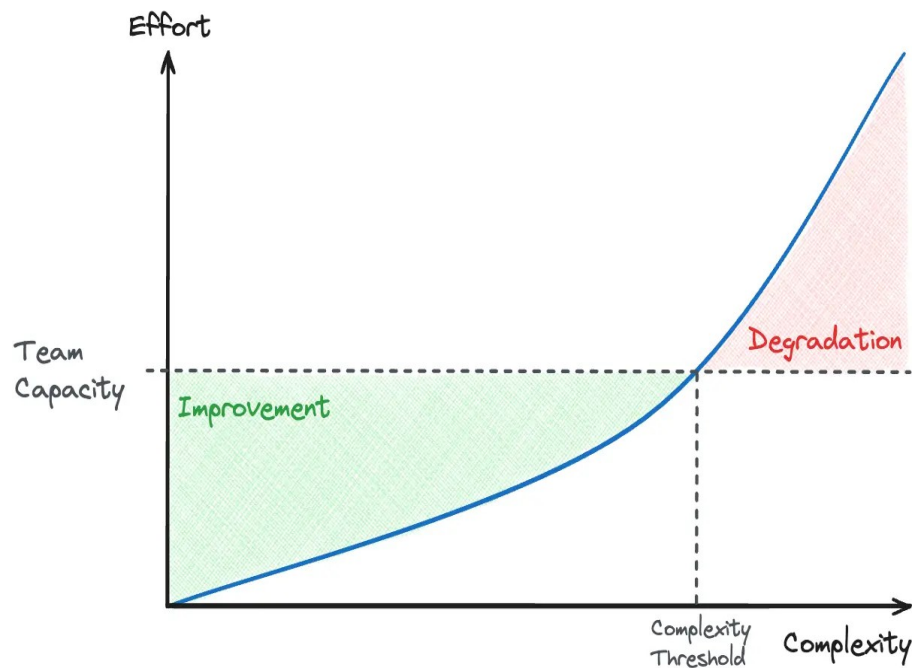


Requisitos de técnicas de desarrollo de software

Principio de diseño KISS

KISS (Keep It Simple, Stupid!)

- El software simple es más fácil de mantener, de evolucionar y es menos propenso a errores.
- Hacer las cosas de la manera más sencilla posible
- Aguantar la tentación de hacer sobreingeniería



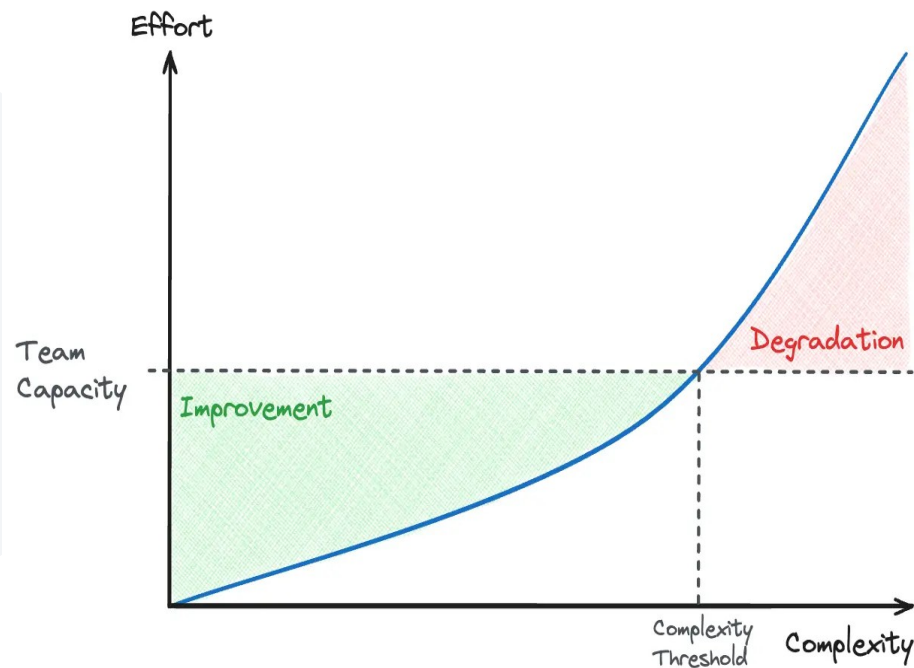
Requisitos de técnicas de desarrollo de software

Principio de diseño SRP

Single Responsibility Principle

- El SRP o principio de responsabilidad única hace que nuestros artefactos de software sean más simples por naturaleza.

“Una clase que se encarga de enviar datos a un tercero, no hagamos que también escriba en nuestra base de datos”



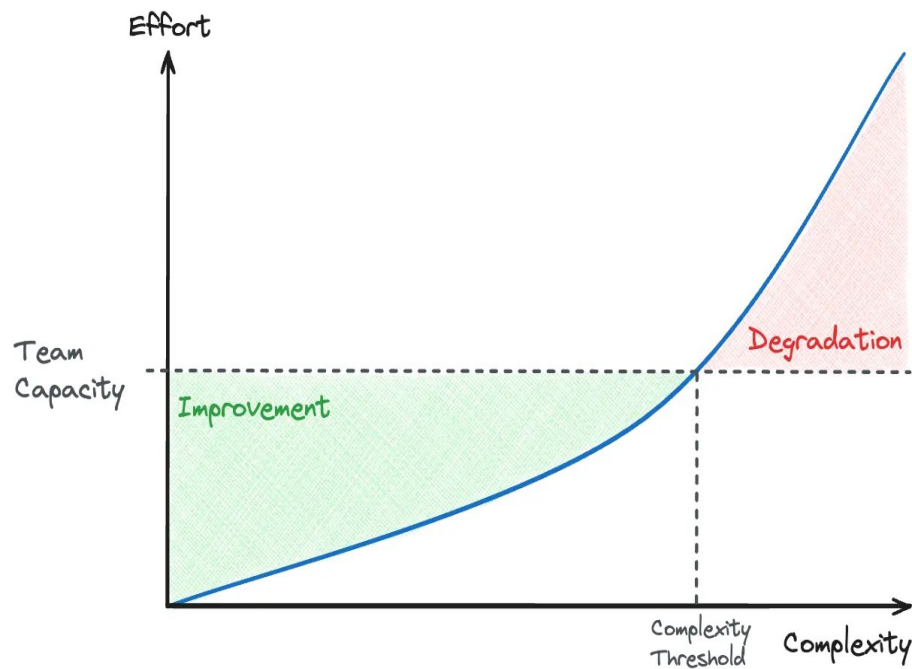
Requisitos de técnicas de desarrollo de software

Principio de diseño YAGNI

YAGNI (You Aren't Gonna Need It)

- Tendemos a hacer las cosas más complejas de lo que deberían ser.
- Tenemos que desarrollar nuestro software sin estar tan pendientes de lo que pueda pasar y hacer foco en lo que queremos conseguir ahora.

No implementar una funcionalidad hasta que realmente sea necesaria.





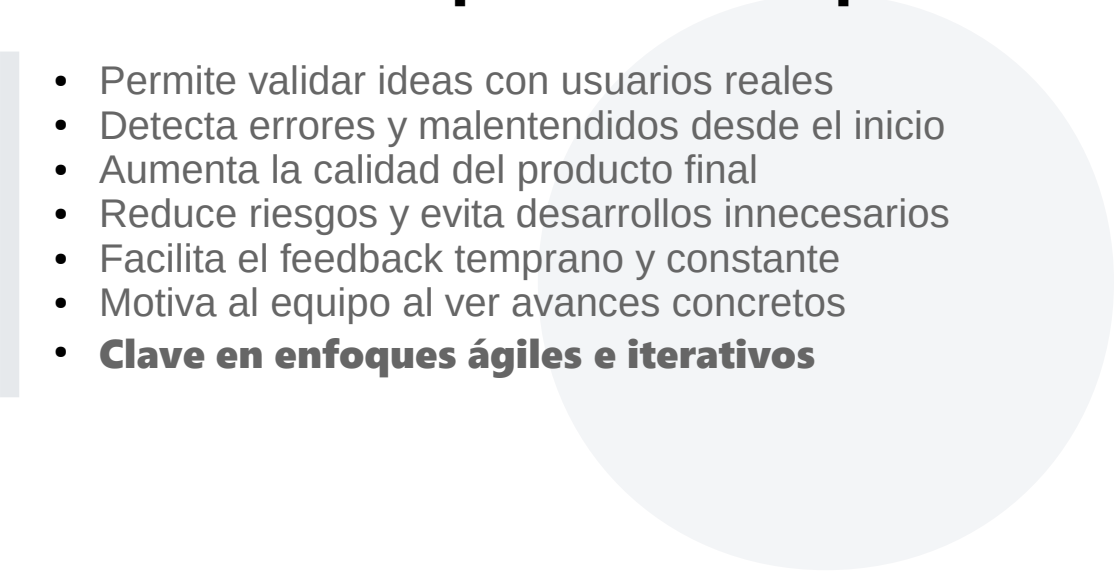
Requisitos de técnicas de desarrollo de software

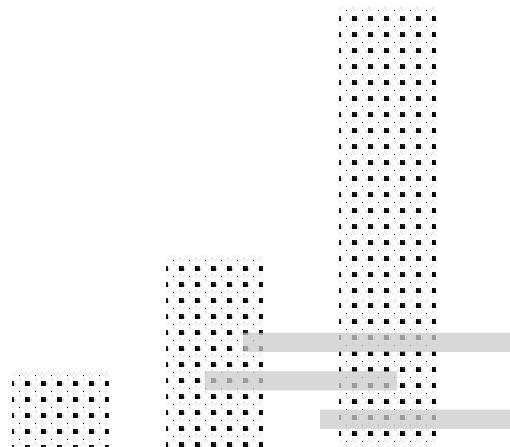
Producir versiones previas lo antes posible



Requisitos de técnicas de desarrollo de software

Producir versiones previas lo antes posible

- 
- Permite validar ideas con usuarios reales
 - Detecta errores y malentendidos desde el inicio
 - Aumenta la calidad del producto final
 - Reduce riesgos y evita desarrollos innecesarios
 - Facilita el feedback temprano y constante
 - Motiva al equipo al ver avances concretos
 - **Clave en enfoques ágiles e iterativos**





Requisitos de técnicas de desarrollo de software

Minimizar el efecto de los cambios

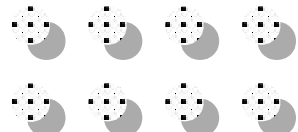
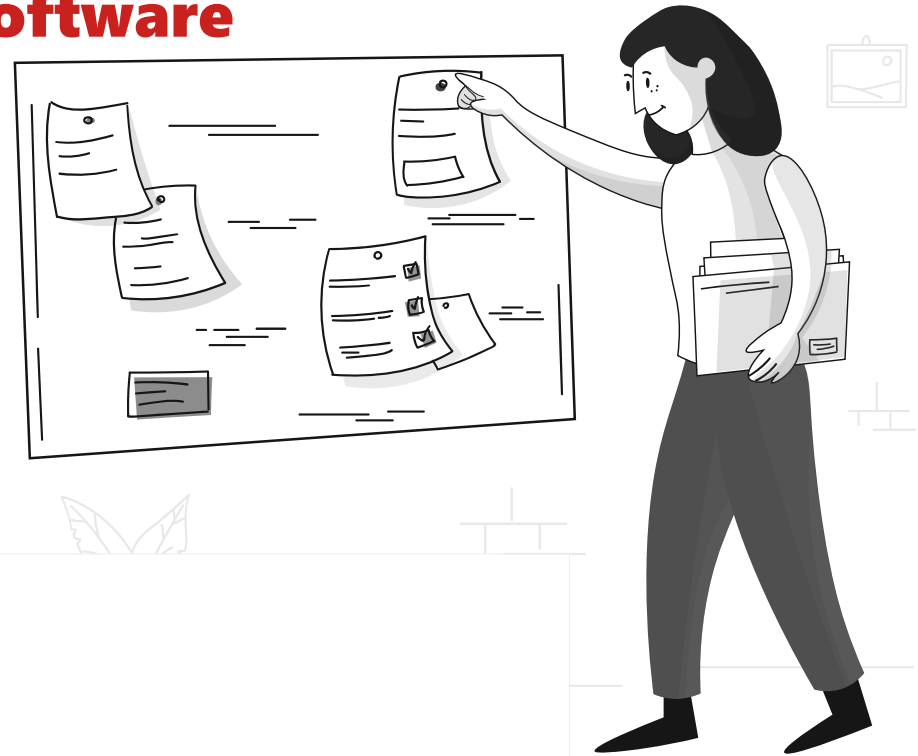
Requisitos de técnicas de desarrollo de software

Minimizar el efecto de los cambios

- El cambio **es inevitable** en cualquier proyecto
- **Buen diseño** limita su impacto en el sistema
- Modificaciones localizadas = menos errores colaterales
- Usa **arquitectura modular** y **bajo acoplamiento**
- Pruebas automatizadas detectan fallos rápidamente
- **Control de versiones** evita perder el control
- **Diseñar para cambiar = diseñar bien**

Definiciones de **Ingeniería del software**

- Estudio y aplicación de enfoques **sistemáticos, disciplinados** y **cuantificables** para la **producción** y **mantenimiento** de software [IEEE]
- Disciplina que comprende todos los aspectos de producción de software desde las etapas iniciales de la **especificación** del sistema, hasta el **mantenimiento** de éste después de su implantación.





Ingeniería del software

Capas de la Ingeniería del software

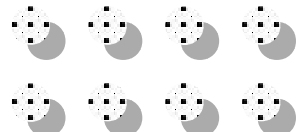
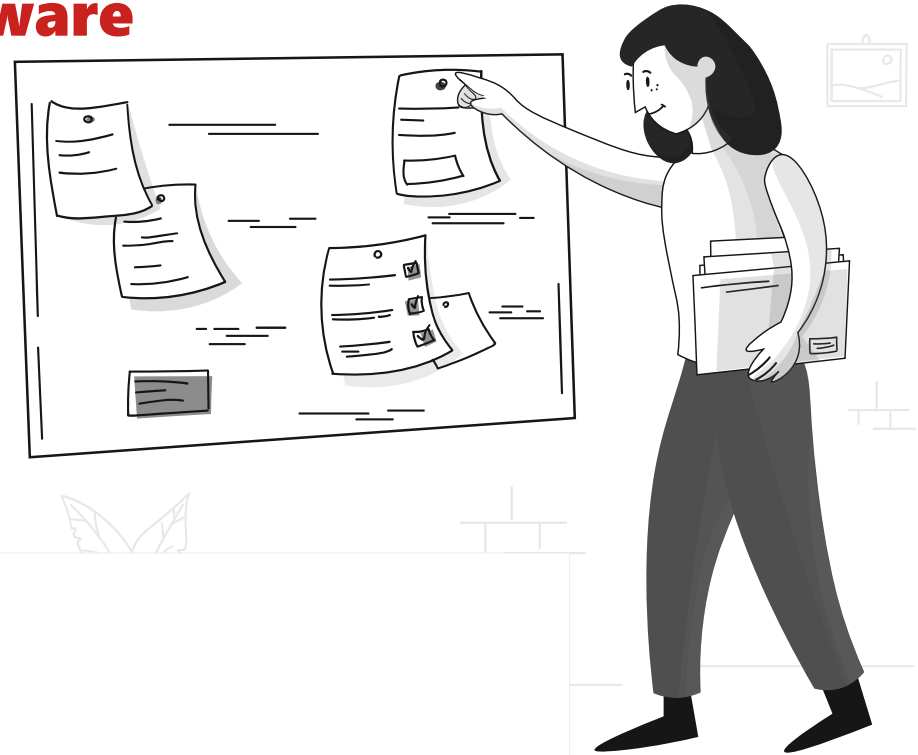
La ingeniería del software se apoya en tres capas fundamentales:

- **Proceso:** organiza las tareas clave del desarrollo y la gestión del proyecto.
- **Métodos:** definen las técnicas y enfoques usados (análisis, diseño, pruebas, etc.).
- **Herramientas:** software que apoya y automatiza las tareas técnicas.

Todas las capas deben estar alineadas para asegurar el éxito del desarrollo: no basta con herramientas potentes sin un proceso bien definido.

Definiciones de **proceso del software**

- Marco de trabajo de las **tareas requeridas para construir software** de calidad
- **Conjunto de fases** por las que pasa el sistema **a lo largo del tiempo**, desde la fase de estudio y concepción hasta la de realización, explotación y mantenimiento.



Fases del **proceso del software**

01

Estudio y Concepción

- Se identifica el problema o necesidad.
- Se analiza si es viable técnica y económicamente.
- Se definen los objetivos generales del sistema.
- Resultado típico: un documento de requisitos inicial

03

Diseño del Sistema

- Se define cómo se va a construir el sistema.
- Se diseñan la arquitectura, las interfaces y los componentes.
- Se establecen decisiones técnicas clave (lenguaje, base de datos, etc.).
- El objetivo es tener una guía clara para programar.

02

Anál. y Espec. de Requisitos

- Se detallan qué debe hacer el sistema.
- Se recoge información del usuario o cliente.
- Se documentan requisitos funcionales y no funcionales.
- Sirve como contrato entre usuarios y desarrolladores.

04

Implementación

- Los programadores escriben el código fuente.
- Se integran módulos y se realiza verificación inicial.
- Se aplican buenas prácticas de codificación.

Fases del **proceso del software**

05

Pruebas

- Se verifica que el sistema funcione correctamente.
- Hay pruebas unitarias, de integración, de sistema y de aceptación.
- Se detectan errores antes de poner el sistema en producción.

07

Mantenimiento

- Se corrigen errores detectados en producción.
- Se adaptan funcionalidades a nuevas necesidades.
- Se mejora el rendimiento o se actualizan tecnologías.
- Es la fase más larga y costosa del ciclo de vida.

06

Explotación (Despliegue)

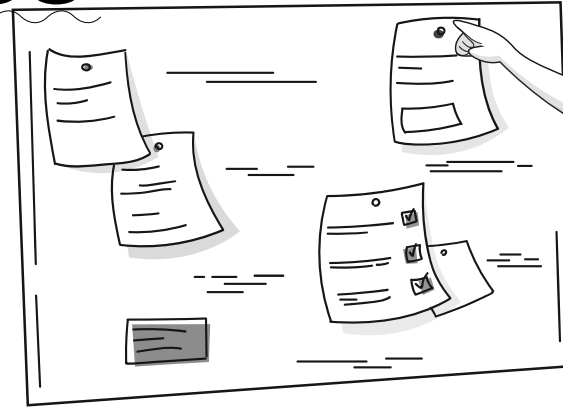
- El sistema entra en funcionamiento real.
- Puede ser usado por usuarios finales.
- Es habitual realizar formación y soporte inicial.

Estas fases **no** siempre se siguen de forma **secuencial**. En metodologías Ágiles, muchas de ellas **se repiten de forma iterativa e incremental**.

Modelos de proceso

Es un marco de referencia que define fases, actividades, entregables y flujo de trabajo para organizar el desarrollo de software. Ayuda a planificar, ejecutar y controlar los proyectos de forma sistemática.

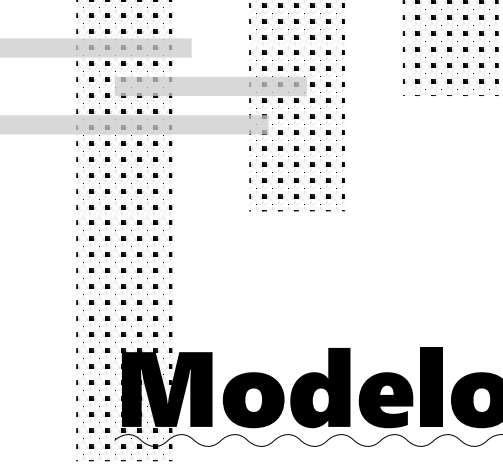
- Modelos lineales
- Modelos iterativos o evolutivos
- Otros modelos



❑ Idea clave:

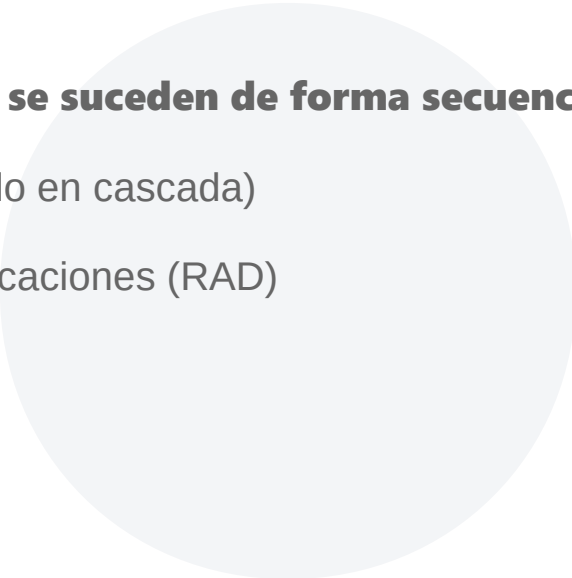
No existe un único modelo perfecto:
cada proyecto requiere adaptar el enfoque según sus objetivos, restricciones, recursos y nivel de incertidumbre.



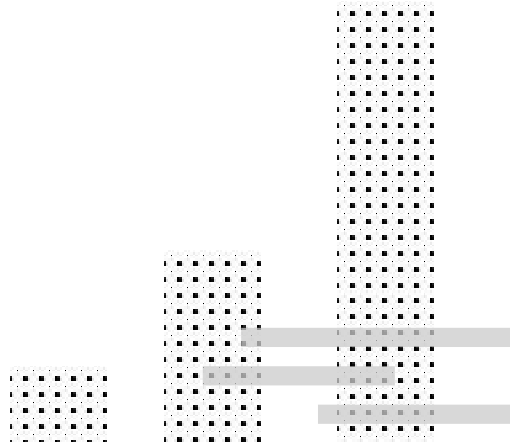


Modelos de proceso

Modelos lineales

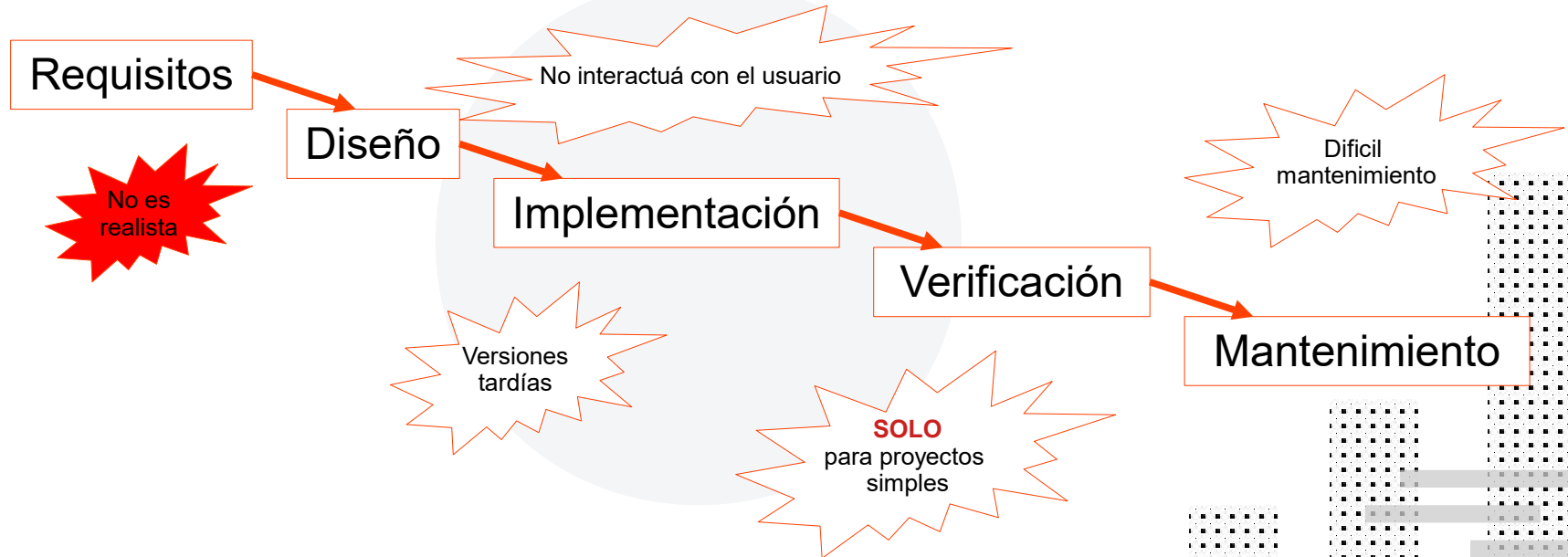


Modelos donde **las etapas se suceden de forma secuencial**, con poco margen para retrocesos.

- Lineal secuencial (modelo en cascada)
 - Prototipos
 - Desarrollo rápido de aplicaciones (RAD)
- 

Modelos de proceso **lineal**

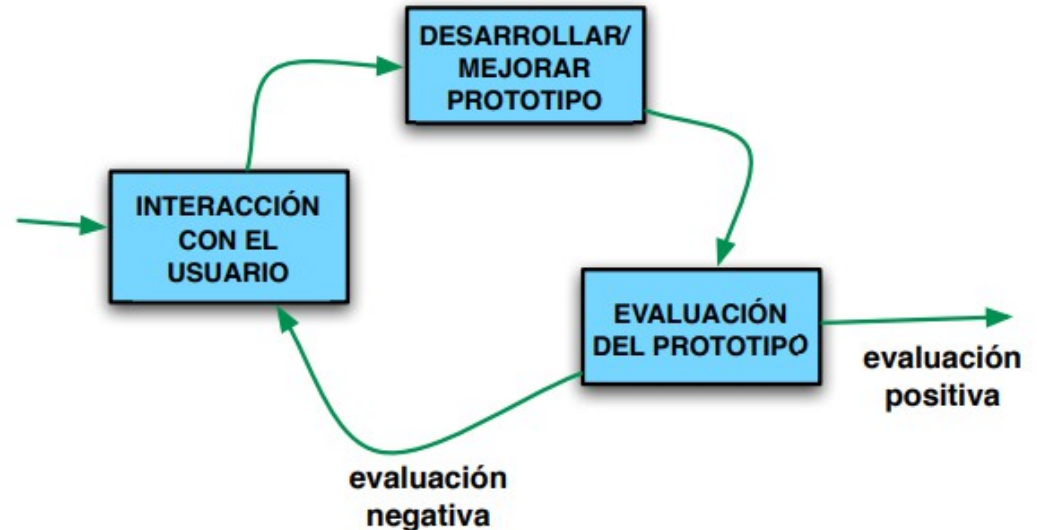
Modelo Lineal secuencial (modelo en cascada)



Modelos de proceso **lineal**

Modelo basado en prototipos

- Un **prototipo** es una versión previa del sistema (de funcionalidad limitada), de la que los usuarios evaluarán el diseño y la información generada.
 - Se crea un modelo funcional preliminar para validar requisitos con el usuario
 - Útil cuando los requisitos no están claros
 - El desarrollo del prototipo es lineal para hacerlo lo más simple y rápido posible



Modelos de proceso **lineal**

Modelo basado en prototipos

- Este modelo debe plantearse como un **método de determinación de requisitos** cuando:
 - El propio usuario no conoce a fondo todo el ámbito, o no es capaz de saber la información que va a necesitar.
 - Los encargados del desarrollo no tienen información ni experiencia.
 - La implantación supone un riesgo
- La idea subyacente: Los usuarios pueden aportar más cosas (requisitos y/o problemas) trabajando sobre un sistema que funciona que sobre un modelo teórico sobre el papel.

Modelos de proceso **lineal**

Modelo basado en prototipos

- Tras la evaluación del prototipo se pueden plantear varias salidas:
 - Mejorar el prototipo
 - Abandonar el proyecto
 - Completar el desarrollo del sistema y... tirar el prototipo
- En cuanto al desarrollo (y modificación), lo interesante es que sea lo más rápido posible. Por ello resulta bastante apropiado disponer de mecanismos de reutilización de código y/o generadores automáticos de aplicaciones.

Modelos de proceso **lineal**

Modelo de desarrollo rápido de aplicaciones (RAD)

El modelo RAD (**Rapid Application Development**) es un enfoque que prioriza la **rapidez** en la construcción de aplicaciones, especialmente mediante **herramientas automáticas de generación de código**, plantillas y reutilización de componentes.

❑ **Características principales**

- Alta velocidad de desarrollo
- Ideal para aplicaciones de tipo comercial o con ciclos cortos.
- Modular y automatizado
- Divide el trabajo en componentes que se pueden desarrollar en paralelo.
- Apoyo en herramientas CASE o generadores de código

Limitaciones

- No adecuado para sistemas complejos o críticos
- Alto riesgo si los requisitos no están bien definidos
- Dependencia de herramientas

Modelos de proceso **lineal**

Modelo de desarrollo rápido de aplicaciones (RAD)



- **Modelado de datos**
 - Se identifican las entidades y relaciones fundamentales.
- **Modelado de procesos**
 - Se definen los flujos y la lógica de negocio.
- **Generación de aplicaciones**
 - Se crean directamente los módulos funcionales, muchas veces usando herramientas de generación visual.
- **Pruebas**
 - Se validan los módulos construidos antes de integrar y entregar.

Modelos de proceso

Modelos iterativos o evolutivos

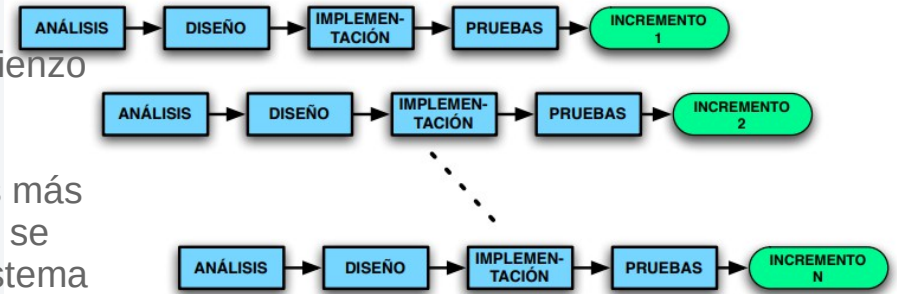
Desarrollan el sistema por ciclos sucesivos, mejorando o ampliando funcionalidades en cada iteración.

- **Incremental:**
 - Se construye el software por bloques funcionales entregados progresivamente
- **Espiral:**
 - Combina desarrollo con gestión de riesgos
 - Cada vuelta en la espiral incluye planificación, análisis, diseño, implementación y evaluación
- **Desarrollo basado en componentes:**
 - Se integran y reutilizan módulos ya existentes o desarrollados independientemente

Modelos de proceso **evolutivo**

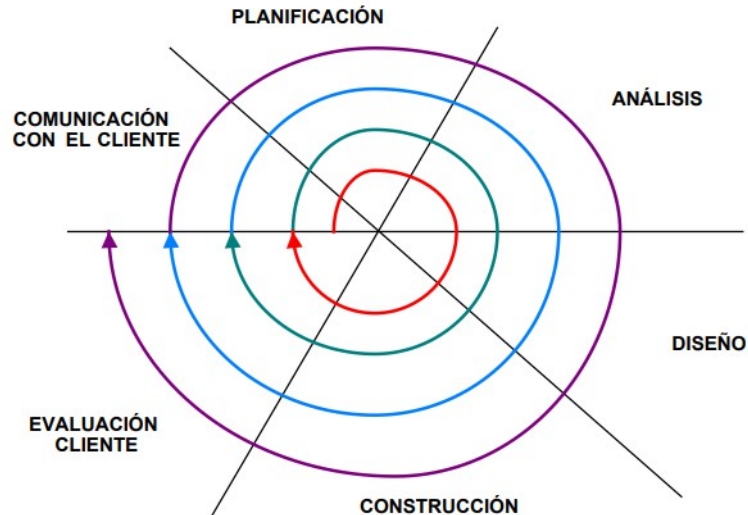
Modelo incremental

- Producto operativo en cada incremento, a diferencia de los prototipos
- Útil si hay problemas para terminar a tiempo
- Los incrementos deben quedar determinados al comienzo del desarrollo.
- Los primeros incrementos deben cubrir los requisitos más importantes (con mayor prioridad) del sistema. Estos se probarán varias veces, aumentando la calidad del sistema



Modelos de proceso **evolutivo**

Modelo en espiral

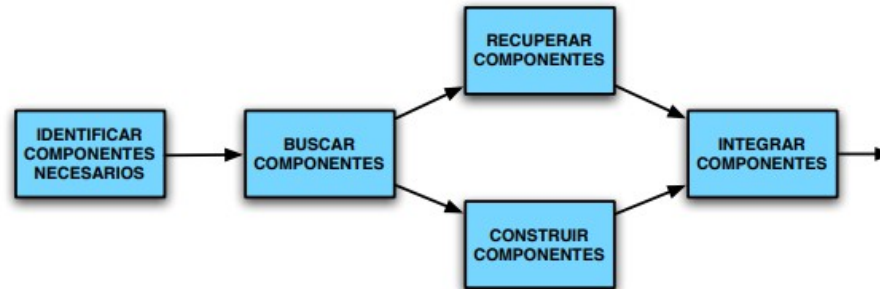


- Combina desarrollo con gestión de riesgos
- Cada vuelta en la espiral incluye planificación, análisis, diseño, implementación y evaluación
- Siempre se trabaja con el sistema completo
- Posibles puntos de entrada
- Activo toda la vida del software
- Difícil de gestionar

Modelos de proceso **evolutivo**

Modelo basado en componentes

- Se plantea como una variante del proceso en espiral
- Se basa en la reutilización
- Se sustituye las tareas de diseño y construcción por la búsqueda y desarrollo de componentes
- Se ahorra trabajo y se gana en rapidez
- Aumenta la calidad del software



Modelos de proceso

Otros modelos

- Desarrollo ágil:
 - Priorizan entregas tempranas, colaboración y adaptación al cambio
 - Ejemplos: Scrum, XP
- Métodos formales:
 - Uso de matemáticas y lógica para especificar, verificar y validar software crítico
 - Ejemplos: B, Z, TLA+

Modelos de proceso: **Otros Modelos**

Desarrollo ágil

- Colaboración del usuario
- Software que funcione antes que documentación
- Respuesta a cambios por encima de seguimiento de planes
- Entregas frecuentes de SW
- Calidad
- Simplicidad

Modelos de proceso: Otros Modelos

Desarrollo ágil: Scrum

❑ ¿Qué es Scrum?

Scrum es un enfoque ágil para la gestión de proyectos que **divide el trabajo en ciclos cortos y repetibles** llamados **sprints**. Favorece la colaboración continua, la entrega temprana de valor y la capacidad de adaptación.

✓ Beneficios de Scrum

- Entrega valor desde las primeras semanas
- Fomenta la transparencia y la mejora continua
- **Facilita la adaptación a cambios**
- Potencia la colaboración y la motivación del equipo
- Riesgos reducidos gracias al **feedback constante**

Cuándo no es adecuado Scrum

- Si los **requisitos** están totalmente cerrados y **no van a cambiar**
- Si **el equipo no es autoorganizado** o hay jerarquía rígida
- Si **el cliente no puede colaborar** con frecuencia

Modelos de proceso: **Otros Modelos**

Métodos formales

- Fundamento: Especificación formal de los requisitos + transformaciones formales de la especificación en código
- Corrección demostrable
- Adecuado para sistemas críticos
- El desarrollo se ralentiza y requiere alta cualificación
- Dificulta la comunicación con el cliente
- Se recomienda la utilización parcial

Ejercicios

1. Suponga que se le ha nombrado jefe de proyecto en una empresa de desarrollo de sistemas de información. El proyecto a desarrollar es bastante similar a otros que ha desarrollado su equipo, aunque mayor y más complejo. Los requisitos han sido detalladamente documentados por el cliente. ¿Qué modelo de proceso elegiría para este proyecto y por qué?

Ejercicios

2. Suponga que se le ha nombrado jefe de proyecto en una pequeña compañía de desarrollo de software. Su proyecto consiste en desarrollar un juego para ordenador que resulte muy innovador.

Como la competencia en el mercado de entretenimiento es muy alta, el proyecto debe estar terminado rápidamente. ¿Qué modelo de proceso elegiría para este proyecto y por qué?

Ejercicios

3. Suponga que se le ha nombrado jefe de proyecto en una gran empresa de productos de software. Su trabajo consiste en dirigir la próxima versión de su famoso procesador de texto. Como la competencia es estricta, se han establecido fechas de entrega estrictas. ¿Qué modelo de proceso elegiría para este proyecto y por qué?

Ejercicios

4. Suponga que se le ha nombrado jefe de un proyecto de software de diagnóstico médico. La máxima prioridad es la fiabilidad, y el tiempo disponible para el desarrollo es abundante. ¿Qué modelo de proceso elegiría para este proyecto y por qué?